

MODULE 05 · DOMAIN 2 - DATA INGESTION AND LOADING

Auto Loader in depth

cloudFiles, checkpoints, file discovery modes, schema inference and evolution, and the rescued data column.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 05 of 36

Auto Loader is a **Structured Streaming source over a directory**. Everything it does follows from that, including the things people find surprising.

The minimal shape

LINE	PURPOSE
<code>spark.readStream</code>	It is a stream, not a batch read
<code>.format("cloudFiles")</code>	The Auto Loader source
<code>.option("cloudFiles.format", "json")</code>	Format of the source files. Required.
<code>.option("cloudFiles.schemaLocation", path)</code>	Where the inferred schema is stored
<code>.load(source_path)</code>	The directory to watch
<code>.writeStream.option("checkpointLocation", p)</code>	Where progress is recorded

ON THE EXAM

Two locations, two jobs. `schemaLocation` holds the schema; `checkpointLocation` holds progress. They may share a directory.

The checkpoint is the memory

- ◆ **It records which files have been processed**, so a restart resumes rather than repeats.
- ◆ **Exactly-once processing** follows from it. This is the guarantee, not a best effort.
- ◆ **Delete the checkpoint and the stream reingests everything**. There is no memory anywhere else.
- ◆ **Every source location needs its own checkpoint**, even when several streams write to the same target table.

ON THE EXAM

Setting `schemaLocation` enables schema inference and evolution. Without it, you get neither.

Auto Loader or a plain streaming file read?

`SPARK.READSTREAM.FORMAT("JSON")`

The generic file source

- Lists the whole directory on every micro-batch
- Cost grows with directory size, not with new data
- No schema evolution handling
- No rescued data column

`FORMAT("CLOUDFILES")`

Auto Loader

- Optimised discovery, and a notification option
- Cost tracks new files, not total files
- Schema inference, hints and evolution modes
- Rescued data column included by default

THE TIEBREAKER

Both are streams with checkpoints. Auto Loader adds efficient discovery and the whole schema apparatus on top.

How Auto Loader finds new files

DIRECTORY LISTING

Ask the storage what is there

- The default mode
- No extra permissions beyond reading the data
- Cost grows with the size of the directory
- Fine for modest volumes and simple setups

FILE NOTIFICATION

Let the storage tell you

- Subscribes to file events from the cloud provider
- Scales to millions of files an hour
- Needs permission to create or use cloud resources
- Databricks recommends it for most workloads

WORTH KNOWING

You can switch between the two across stream restarts and still keep exactly-once processing guarantees.

Classic file notification mode

- ◆ **Set `cloudFiles.useNotifications` to `true`** and Auto Loader sets up the notification and queue services itself.
- ◆ **It creates cloud resources** — an SNS topic and SQS queue on AWS, equivalents on Azure and GCP.
- ◆ **Each stream manages its own queue.** Many streams on one bucket means many parallel notification pipelines.
- ◆ **Notification systems do not guarantee 100% delivery**, which is what `cloudFiles.backfillInterval` exists for.

ON THE EXAM

`backfillInterval` triggers an asynchronous backfill at a set interval, so anything a missed notification skipped still gets processed.

File notification with file events

The managed option, and the current recommendation

- ◆ **Enable file events on the external location**, then set `cloudFiles.useManagedFileEvents` to `true`.
- ◆ **One managed service, shared**. A single topic and queue serve many consumers instead of one each.
- ◆ **The first run does a full directory listing** to synchronise, then reads from the file events cache.
- ◆ **Run the stream at least every seven days**. Otherwise the stored read position expires and a full listing is needed again.

ON THE EXAM

Databricks recommends migrating from directory listing to file notification with file events for most workloads.

The three discovery configurations

CONFIGURATION	OPTION	WHEN
Directory listing	(the default)	Modest volumes, no setup, quick to start
Classic file notification	<code>cloudFiles.useNotifications</code>	High volume, per-stream queues
File events	<code>cloudFiles.useManagedFileEvents</code>	High volume, managed and shared. Recommended.

ON THE EXAM

`cloudFiles.useIncrementalListing` is deprecated and defaults to false from DBR 17.3 LTS. Ignore it in older study material.

Schema **inference** happens once, when the stream first runs. Schema **evolution** is the ongoing policy for what happens when the data changes afterwards.

Schema inference

- ◆ **Set `cloudFiles.schemaLocation`** and Auto Loader infers a schema on first run and stores it there.
- ◆ **For JSON and CSV, columns are inferred as strings by default.** This is deliberate, and it surprises everyone.
- ◆ **Why:** a column that looks like an integer in the first files may contain text later. String never loses data.
- ◆ **The schema location may be the same directory as the checkpoint.** In Lakeflow pipelines both are managed for you.

ON THE EXAM

'Why did everything arrive as STRING?' is one of the most reliably asked questions about Auto Loader.

Two ways to get real types

- ◆ **cloudFiles.inferColumnTypes = true.** Infer actual types rather than defaulting to string. Default is false.
- ◆ **cloudFiles.schemaHints.** Name specific columns and their types, and let the rest be inferred.
- ◆ **Hints are surgical.** Fix the three columns you care about without committing on the other forty.
- ◆ **A hint still permits addNewColumns,** whereas supplying a full schema does not.

ON THE EXAM

Prefer hints when you know some columns and not others. Prefer inferColumnTypes when you trust the data broadly.

The rescued data column captures anything that did not fit — wrong types, unexpected fields, casing mismatches — as JSON, **instead of dropping it.**

The four schema evolution modes

MODE	BEHAVIOUR WHEN A NEW COLUMN APPEARS
<code>addNewColumns</code>	Stream fails. The new column is added to the schema. Existing column types are unchanged.
<code>rescue</code>	Schema never evolves and the stream never fails. New columns are recorded in the rescued data column.
<code>failOnNewColumns</code>	Stream fails, and will not restart until the schema is updated or the offending file removed.
<code>none</code>	Schema is not evolved and new columns are ignored. Not rescued unless the option is set. Stream does not fail.

ON THE EXAM

Learn all four rows. Questions describe the behaviour and ask for the mode.

The defaults, which are not what you would guess

- ◆ `addNewColumns` is the default when you do not provide a schema.
- ◆ `none` is the default when you do provide a schema.
- ◆ `addNewColumns` is not allowed at all when a full schema is provided.
- ◆ It does work if you supply your schema as a schema hint instead.

ON THE EXAM

Two different defaults depending on whether you supplied a schema. That asymmetry is exactly the sort of thing an exam plants.

What 'the stream fails' actually means

- ◆ Auto Loader detects the new column and stops with an `UnknownFieldException`.
- ◆ Before it stops, it updates the schema location with the new column merged onto the end.
- ◆ On restart, the evolved schema is used and processing continues normally.
- ◆ Databricks recommends running Auto Loader streams from Lakeflow Jobs configured to restart automatically.

ON THE EXAM

With `addNewColumns` the failure is a feature, not a fault. One deliberate stop, then the pipeline carries on with the wider schema.

Two more evolution behaviours

- ◆ **A renamed column is treated as a new column.** The old one is filled with NULL for new rows.
- ◆ **A dropped column is a soft delete.** New rows get NULL; existing data is untouched.
- ◆ **addNewColumnsWithTypeWidening** also evolves on compatible type changes, from Databricks Runtime 16.4.
- ◆ **Widening is lossless only:** int to long, float to double, and similar promotions.

ON THE EXAM

Type widening is newer than most study material. It is a fifth mode name you may not have seen before.

Triggers: how often the stream runs

- ◆ **Trigger.AvailableNow** processes everything available, then stops. Batch behaviour with streaming bookkeeping.
- ◆ **Trigger with a processing time** runs micro-batches on an interval, leaving the cluster up between them.
- ◆ **Continuous processing** means a cluster that never stops, and a bill that never stops either.
- ◆ **AvailableNow on a schedule is the common production pattern** for data that arrives in bursts.

ON THE EXAM

AvailableNow gives you a checkpoint's exactly-once guarantee with a batch job's cost profile. That combination is why it dominates.

Three options that decide what gets picked up

- ◆ **cloudFiles.includeExistingFiles.** Default true. Whether files already present are processed, or only new arrivals.
- ◆ **It is evaluated only on the first start.** Changing it after the stream has run has no effect at all.
- ◆ **cloudFiles.backfillInterval.** Periodic asynchronous backfill, insuring against missed notifications.
- ◆ **cloudFiles.allowOverwrites.** Default false. Whether a changed file is reprocessed.

ON THE EXAM

Like COPY INTO, Auto Loader will not reprocess a modified file by default. Same principle, different mechanism.

Auto Loader without writing Python

- ◆ **Inside Lakeflow Spark Declarative Pipelines**, schema location and checkpoints are managed for you.
- ◆ **As a streaming table in Databricks SQL**, which wraps Auto Loader in a SQL declaration.
- ◆ **Via `read_files` in SQL**, the table-valued function over files in storage.
- ◆ **Same engine underneath**. The options you have learnt still apply, just expressed differently.

ON THE EXAM

A SQL-first team is not shut out of Auto Loader. Streaming tables are Databricks' recommended route for them.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Every JSON column arrived as STRING	Default inference. Use <code>inferColumnTypes</code> or <code>schemaHints</code> .
The pipeline must never stop, but capture new fields	<code>rescue</code>
A new column must be added automatically	<code>addNewColumns</code> , with a job that restarts
Schema drift must halt the pipeline immediately	<code>failOnNewColumns</code>
Millions of files an hour	File notification, ideally with file events
Data arrives in bursts a few times a day	<code>Trigger.AvailableNow</code> on a schedule
Duplicates appeared after a restart	The checkpoint was deleted or moved

ON THE EXAM

Seven lines. If you can answer all seven without hesitating, you have the most valuable module in domain two.

A team sets up Auto Loader against a directory of JSON event files. They set `cloudFiles.format` and `cloudFiles.schemaLocation`, and start the stream. It runs without error.

An analyst then reports that she cannot do arithmetic on the `quantity` or `unit_price` columns. Inspecting the bronze table, every single column has type `STRING`, including several that are obviously numeric.

What happened, and what is the most appropriate fix?

- A The JSON files are malformed; the source system must be corrected
- B Auto Loader infers JSON columns as `STRING` by default; set `cloudFiles.inferColumnTypes` or supply `schemaHints`
- C The `schemaLocation` was not writable, so Auto Loader fell back to a default all-`STRING` schema
- D The stream needs `cloudFiles.schemaEvolutionMode` set to `addNewColumns` to detect types

◆ PAUSE AND DECIDE

What happened, and what is the most appropriate fix?

- A

 The JSON files are malformed
The stream ran without error and produced every expected column. Malformed input would have surfaced in the rescued data column instead.
- B

 Default inference; use inferColumnTypes or schemaHints
Correct. For JSON and CSV, columns are inferred as STRING unless cloudFiles.inferColumnTypes is set to true or hints are supplied.
- C

 schemaLocation was not writable
Invented failure mode. An unwritable schema location produces an error, not a silent fallback.
- D

 Set schemaEvolutionMode to addNewColumns
Confuses evolution with inference. Evolution governs what happens when data changes later; it has no bearing on initial types.

THE SIGNAL

The stream ran without error. That is the clue: nothing failed, the default simply is not what the team expected.

An Auto Loader stream has run reliably for eight months. On Sunday night it stopped with an `UnknownFieldException`. Investigating on Monday, an engineer finds that the source system began sending a new field, `loyalty_tier`, on Sunday evening.

He also notices that the schema stored in the schema location already contains `loyalty_tier`, even though the stream failed.

What is going on?

- A The schema location has been corrupted, and should be deleted so the schema can be re-inferred
- B This is `addNewColumns` behaving as designed: it updated the schema, then stopped. Restarting will resume with the wider schema
- C Someone manually edited the schema location, which caused the stream to fail
- D The mode must be `failOnNewColumns`, so the schema must be updated by hand before restarting

◆ PAUSE AND DECIDE

What is going on?

- A

 Corrupted; delete the schema location
Actively harmful. Deleting it discards the evolved schema, and if the checkpoint goes with it the stream reingests everything.
- C

 Someone edited the schema by hand
Invents a human. Auto Loader wrote that entry itself, which is exactly the documented sequence.
- B

 addNewColumns working as designed; restart resumes
Correct. It updates the schema location with the new column merged on, then stops. The restart picks up the evolved schema.
- D

 failOnNewColumns, needing a manual schema update
Under failOnNewColumns the schema would not have been updated. The presence of loyalty_tier rules this mode out.

THE SIGNAL

The updated schema is the evidence. failOnNewColumns would have failed without writing anything; addNewColumns writes, then stops.

A payments team ingests transaction files from twelve partner banks. Partners occasionally add fields without warning. The pipeline feeds a regulatory report and must not stop.

The compliance team requires that any unexpected field is retained and reviewable, not discarded, but must never enter the reported dataset without approval.

Which schema evolution mode fits?

☐ A addNewColumns

☐ B failOnNewColumns

☐ C rescue

☐ D none

◆ PAUSE AND DECIDE

Which schema evolution mode fits?

- A

addNewColumns
Fails the stream on every new field, breaking the must-not-stop requirement. It also admits the field to the dataset without approval.
- B

failOnNewColumns
Stops the pipeline and will not restart until a human intervenes. The strictest possible violation of must-not-stop.
- C

rescue
Correct. The schema never evolves and the stream never fails. Unexpected fields land in the rescued data column, retained and reviewable.
- D

none
The stream does not fail, but new columns are ignored and not rescued unless separately configured. Retention fails.

THE SIGNAL

Three requirements: never stop, never discard, never auto-admit. Only rescue satisfies all three at once.

A logistics platform lands roughly 4 million small files per hour into one S3 prefix. An Auto Loader stream in directory listing mode is falling further behind each day, and cloud costs for LIST operations have become the largest line on the bill.

The team has permission to enable file events on the external location.

What should they change?

- ☐ A Increase the cluster size so the stream can process files faster
- ☐ B Switch to file notification mode using file events
- ☐ C Set `cloudFiles.includeExistingFiles` to false to skip the backlog
- ☐ D Replace Auto Loader with a scheduled COPY INTO statement

◆ PAUSE AND DECIDE

What should they change?

- A

 Increase the cluster size
The bottleneck is discovery, not processing. More executors do not reduce the number of LIST calls against the prefix.
- B

 File notification using file events
Correct. Notification removes listing entirely, scales to millions of files an hour, and is Databricks' recommendation over directory listing.
- C

 includeExistingFiles = false
Evaluated only on first start, so it does nothing to a running stream. It also abandons the backlog rather than processing it.
- D

 Scheduled COPY INTO
COPY INTO uses directory listing only, so it has the same bottleneck in a less capable tool. Strictly backwards.

THE SIGNAL

LIST costs being the largest line on the bill names the bottleneck outright. The fix has to stop the listing.

Two Auto Loader streams ingest into the same bronze table: one from a European partner's directory, one from an American partner's. To keep configuration simple, an engineer points both streams at the same `checkpointLocation`.

Records begin going missing. Neither stream reports an error.

What is wrong?

- ☐ A Two streams cannot write to the same Delta table concurrently
- ☐ B Each source location requires its own checkpoint; sharing one means the streams overwrite each other's progress
- ☐ C The `schemaLocation` must differ, but sharing a `checkpointLocation` is supported
- ☐ D The streams need `cloudFiles.allowOverwrites` set to true

◆ PAUSE AND DECIDE

What is wrong?

- A

 Two streams cannot write to one Delta table
They can. Delta handles concurrent appends, and multiple sources feeding one bronze table is a normal pattern.
- B

 Each source location needs its own checkpoint
Correct. Databricks states this explicitly: if more than one source location loads into the same target, each workload needs a separate streaming checkpoint.
- C

 Only schemaLocation must differ
Half right, which makes it dangerous. Both need to be separate, and the checkpoint is the one causing lost records here.
- D

 Set allowOverwrites to true
Controls whether a modified file is reprocessed. Unrelated to two streams sharing progress state.

THE SIGNAL

Silent data loss with no error is the signature of shared state. Each stream believes the other's progress is its own.

Auto Loader is a stream. The checkpoint is its memory, the schema location is its expectations, and the evolution mode is **what it does when reality disagrees.**